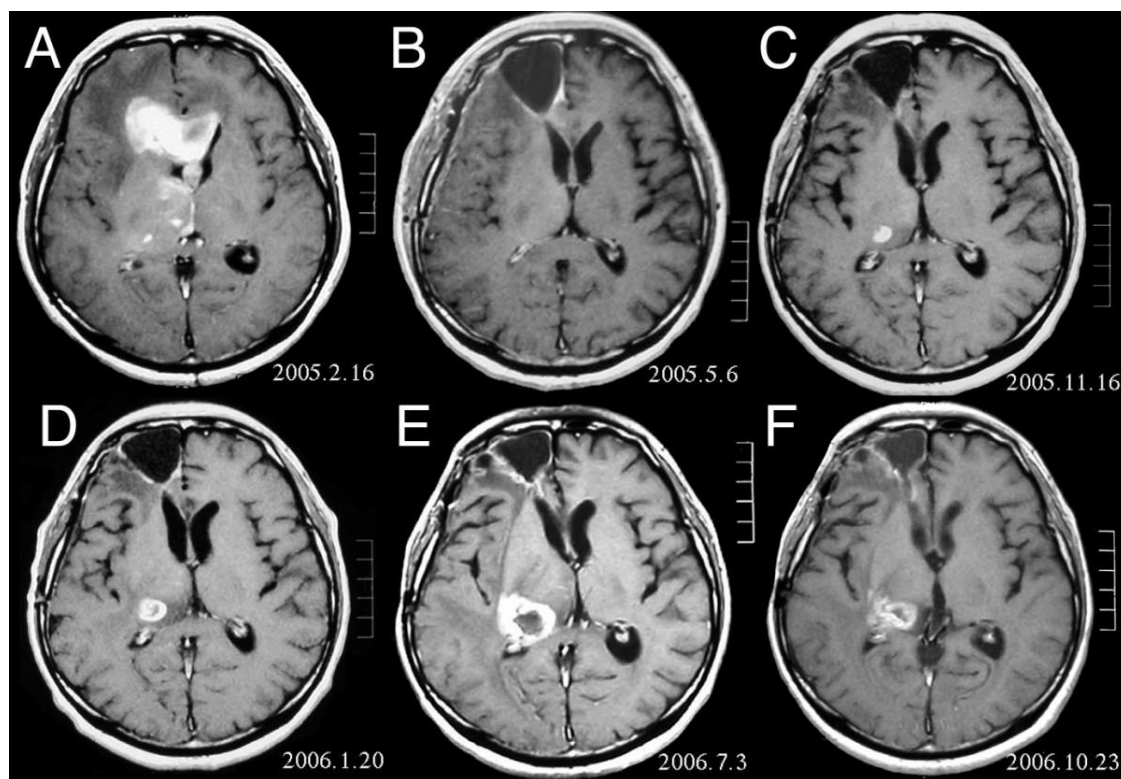


Brain Tumor Classification using Student Teacher Architecture



```
import pandas as pd
import numpy as np
import os

root_path = '/kaggle/input/brisc2025/brisc2025/classification_task/train'

image_paths = []
labels = []

for label in os.listdir(root_path):
    label_path = os.path.join(root_path, label)
    if os.path.isdir(label_path):
        for img_file in os.listdir(label_path):
            if img_file.lower().endswith(('.png', '.jpg', '.jpeg')):
                image_paths.append(os.path.join(label_path, img_file))
                labels.append(label)

df = pd.DataFrame({'image_path': image_paths, 'label': labels})

df
```

	image_path	label
0	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary

```

1      /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
2      /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
3      /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
4      /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
...
4995   /kaggle/input/brisc2025/brisc2025/classificati...  glioma
4996   /kaggle/input/brisc2025/brisc2025/classificati...  glioma
4997   /kaggle/input/brisc2025/brisc2025/classificati...  glioma
4998   /kaggle/input/brisc2025/brisc2025/classificati...  glioma
4999   /kaggle/input/brisc2025/brisc2025/classificati...  glioma

```

```
[5000 rows x 2 columns]
```

```
df.shape
```

```
(5000, 2)
```

```
df.columns
```

```
Index(['image_path', 'label'], dtype='object')
```

```
df.duplicated().sum()
```

```
0
```

```
df.isnull().sum()
```

```
image_path    0
```

```
label         0
```

```
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5000 entries, 0 to 4999
```

```
Data columns (total 2 columns):
```

#	Column	Non-Null Count	Dtype
0	image_path	5000 non-null	object
1	label	5000 non-null	object

```
dtypes: object(2)
```

```
memory usage: 78.3+ KB
```

```
df['label'].unique()
```

```
array(['pituitary', 'no_tumor', 'meningioma', 'glioma'], dtype=object)
```

```
df['label'].value_counts()
```

```
label
```

```
pituitary    1457
```

```
meningioma   1329
```

```
glioma       1147
```

```

no_tumor      1067
Name: count, dtype: int64

import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Tumor Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

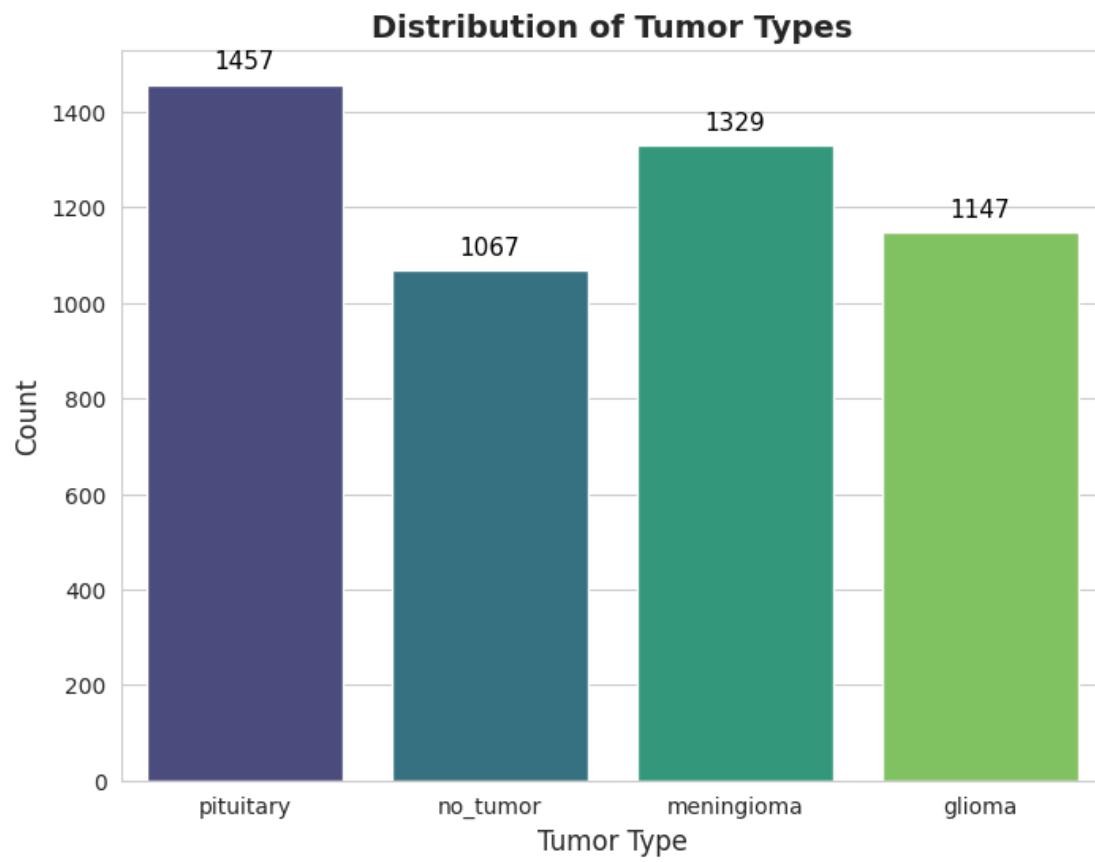
fig, ax = plt.subplots(figsize=(10, 8))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
        startangle=140, colors=colors, textprops={'fontsize': 8, 'weight':
        'bold'},
        wedgeprops={'edgecolor': 'black', 'linewidth': 1})

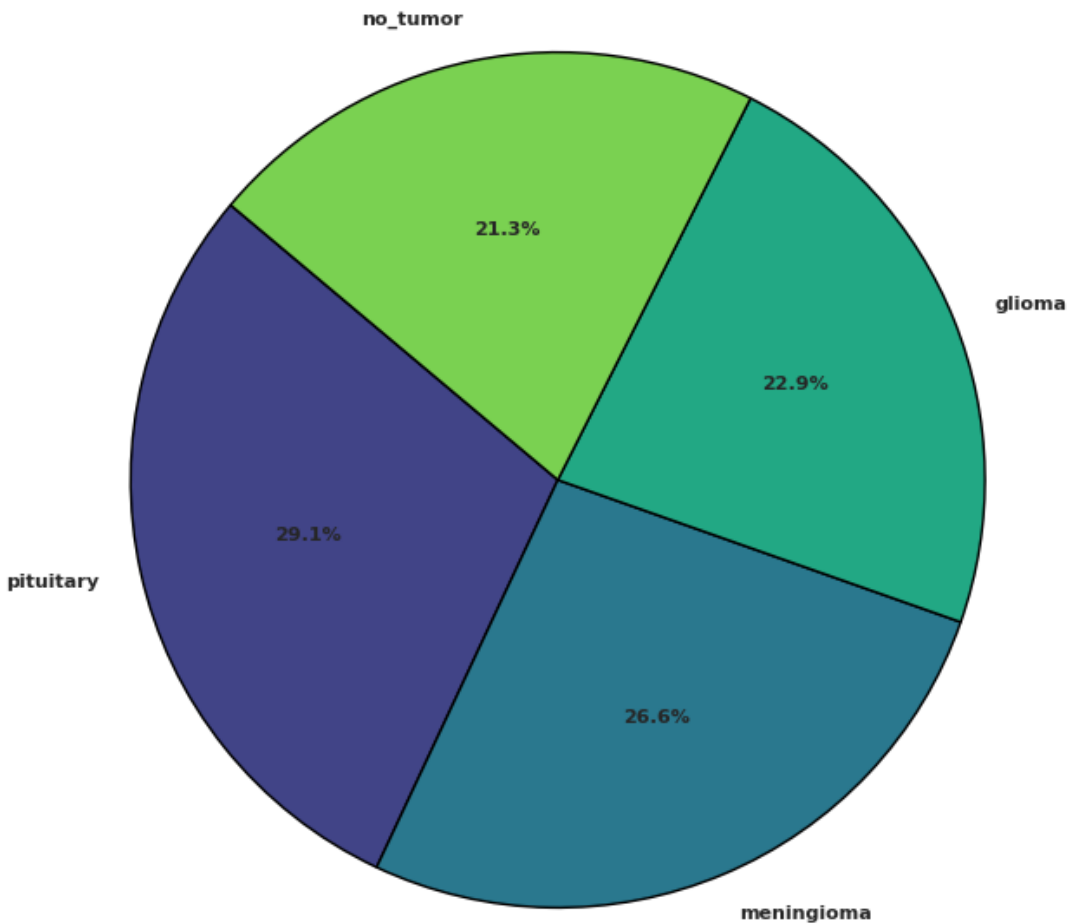
ax.set_title("Distribution of Tumor Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()

```



Distribution of Tumor Types - Pie Chart



```
from PIL import Image

num_images = 5

unique_labels = df['label'].unique()

plt.figure(figsize=(15, len(unique_labels) * 3))

for row_idx, label in enumerate(unique_labels):

    label_images = df[df['label'] ==
label].head(num_images)['image_path'].tolist()

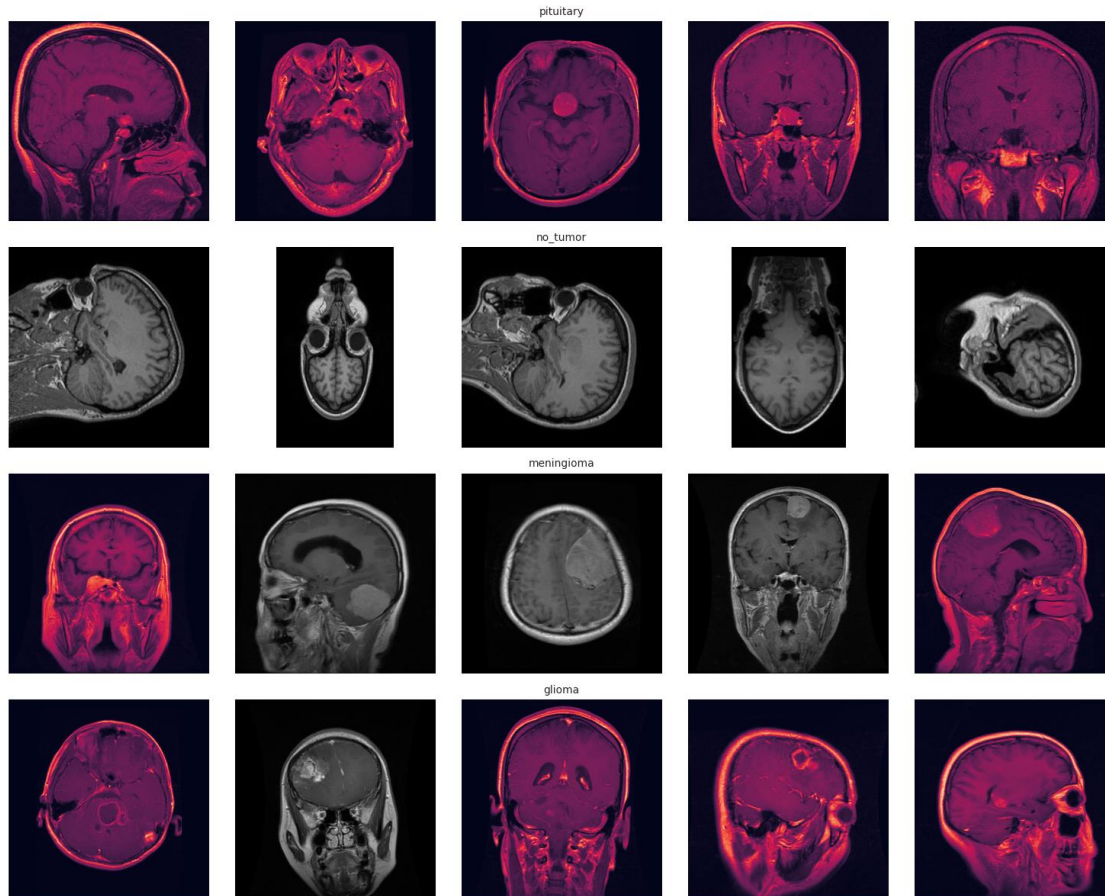
    for col_idx, img_path in enumerate(label_images):
        plt_idx = row_idx * num_images + col_idx + 1
```

```

plt.subplot(len(unique_labels), num_images, plt_idx)
img = Image.open(img_path)
plt.imshow(img)
plt.axis('off')
if col_idx == 2:
    plt.title(label, fontsize=10)

plt.tight_layout()
plt.show()

```



```

import warnings
warnings.filterwarnings('ignore')

max_samples = df['label'].value_counts().max()

balanced_df = df.groupby('label', group_keys=False).apply(
    lambda x: x.sample(n=max_samples, replace=True, random_state=42)
).reset_index(drop=True)

balanced_df = balanced_df[['image_path', 'label']]

df = balanced_df

```

df

	image_path	label
0	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
1	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
2	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
3	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
4	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
...	...	...
5823	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
5824	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
5825	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
5826	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
5827	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary

[5828 rows x 2 columns]

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns
```

```
class JGEKDLoss(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, teacher_logits, student_logits):
        teacher_probs = torch.softmax(teacher_logits, dim=1)
        student_probs = torch.softmax(student_logits, dim=1)

        teacher_joint_graph = torch.bmm(teacher_probs.unsqueeze(2),
teacher_probs.unsqueeze(1))
        student_joint_graph = torch.bmm(student_probs.unsqueeze(2),
student_probs.unsqueeze(1))

        teacher_flat = teacher_joint_graph.view(teacher_joint_graph.size(0),
-1)
        student_flat = student_joint_graph.view(student_joint_graph.size(0),
-1)

        epsilon = 1e-12
        log_student_flat = torch.log(student_flat + epsilon)
```

```

kd_loss = -torch.sum(teacher_flat * log_student_flat, dim=1).mean()
return kd_loss

class BrainTumorDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.dataframe = dataframe
        self.transform = transform
        self.class_names = ['glioma', 'meningioma', 'no_tumor', 'pituitary']
        self.label_map = {name: i for i, name in enumerate(self.class_names)}

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, idx):
        img_path = self.dataframe.iloc[idx]['image_path']

        try:
            image = Image.open(img_path).convert('RGB')
        except FileNotFoundError:
            image = Image.fromarray(np.uint8(np.random.rand(224, 224, 3) *
255))

        label_name = self.dataframe.iloc[idx]['label']
        label = self.label_map[label_name]

        if self.transform:
            image = self.transform(image)

        return image, label

def get_data_loaders(df):

    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
    ])

    if len(df) < 5:
        train_df = df
        test_df = df
    else:
        train_df, test_df = train_test_split(df, test_size=0.2,
random_state=42, stratify=df['label'])

    train_dataset = BrainTumorDataset(train_df, transform)
    test_dataset = BrainTumorDataset(test_df, transform)

```

```
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

```
return train_loader, test_loader, train_dataset.class_names
```

```
def train_and_validate(student_model, teacher_model, train_loader,
test_loader, optimizer, ce_loss_fn, kd_loss_fn, lambda_weight, epochs,
device):
```

```
    train_losses = []
    test accuracies = []
```

```
    for epoch in range(epochs):
```

```
        student_model.train()
        running_loss = 0.0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
```

```
            with torch.no_grad():
                teacher_logits = teacher_model(images)
```

```
            student_logits = student_model(images)
```

```
            ce_loss = ce_loss_fn(student_logits, labels)
            kd_loss = kd_loss_fn(teacher_logits, student_logits)
```

```
            total_loss = ce_loss + lambda_weight * kd_loss
```

```
            optimizer.zero_grad()
            total_loss.backward()
            optimizer.step()
```

```
            running_loss += total_loss.item() * images.size(0)
```

```
    epoch_train_loss = running_loss / len(train_loader.dataset)
    train_losses.append(epoch_train_loss)
```

```
    student_model.eval()
```

```
    correct = 0
```

```
    total = 0
```

```
    with torch.no_grad():
```

```
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = student_model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
```

```

        epoch_test_accuracy = 100 * correct / total
        test accuracies.append(epoch_test_accuracy)

        print(f"Epoch {epoch+1}/{epochs}, Total Train Loss:
{epoch_train_loss:.4f}, Test Accuracy: {epoch_test_accuracy:.2f}%")

    return train_losses, test accuracies

def evaluate_metrics(student_model, test_loader, device, class_names):

    student_model.eval()
    all_labels = []
    all_preds = []

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = student_model(images)
            _, predicted = torch.max(outputs.data, 1)

            all_labels.extend(labels.cpu().numpy())
            all_preds.extend(predicted.cpu().numpy())

    print("\n" + "="*50)
    print("CLASSIFICATION REPORT")
    print("="*50)
    print(classification_report(all_labels, all_preds,
target_names=class_names, zero_division=0))

    cm = confusion_matrix(all_labels, all_preds)

    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.title('Confusion Matrix for Student Model')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.show()

def plot_metrics(train_losses, test accuracies):

    epochs_range = range(1, len(train_losses) + 1)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, train_losses, label='Total Training Loss')
    plt.title('Loss vs. Epoch')

```

```

plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(epochs_range, test_accuracies, label='Test Accuracy')
plt.title('Accuracy vs. Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()

plt.tight_layout()
plt.show()

df['label'] = pd.Categorical(df['label'], categories=['glioma', 'meningioma',
'no_tumor', 'pituitary'])

train_loader, test_loader, CLASS_NAMES = get_data_loaders(df)
NUM_CLASSES = len(CLASS_NAMES)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

teacher_model =
models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
teacher_model.fc = nn.Linear(teacher_model.fc.in_features, NUM_CLASSES)
teacher_model.to(DEVICE)
teacher_model.eval()
for param in teacher_model.parameters():
    param.requires_grad = False

student_model =
models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
student_model.fc = nn.Linear(student_model.fc.in_features, NUM_CLASSES)
student_model.to(DEVICE)

ce_loss_fn = nn.CrossEntropyLoss()
kd_loss_fn = JGEKDLoss()
optimizer = optim.Adam(student_model.parameters(), lr=1e-4)
lambda_weight = 0.5
EPOCHS = 10

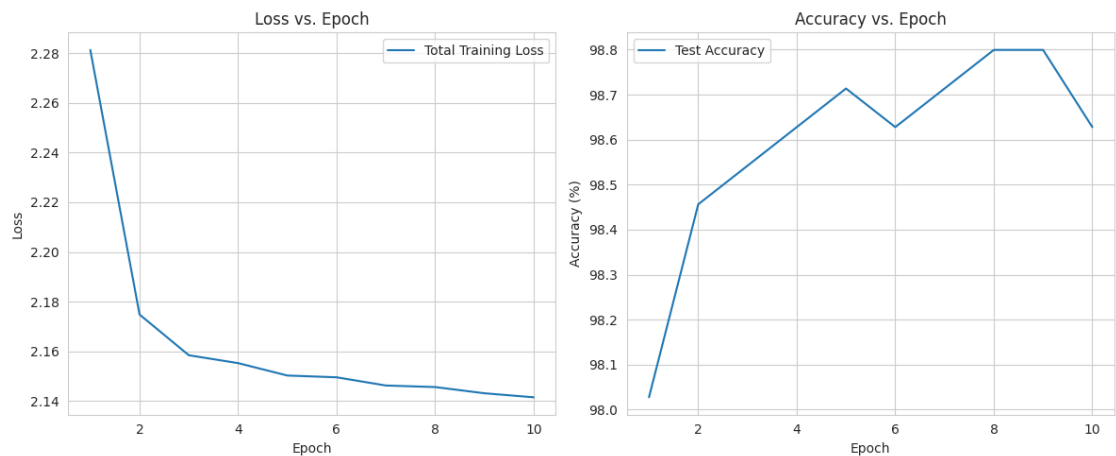
print(f"Starting JGEKD training for Student model on {DEVICE} for {EPOCHS}
epochs.")
train_losses, test_accuracies = train_and_validate(
    student_model, teacher_model, train_loader, test_loader,
    optimizer, ce_loss_fn, kd_loss_fn, lambda_weight, EPOCHS, DEVICE
)

plot_metrics(train_losses, test_accuracies)
evaluate_metrics(student_model, test_loader, DEVICE, CLASS_NAMES)

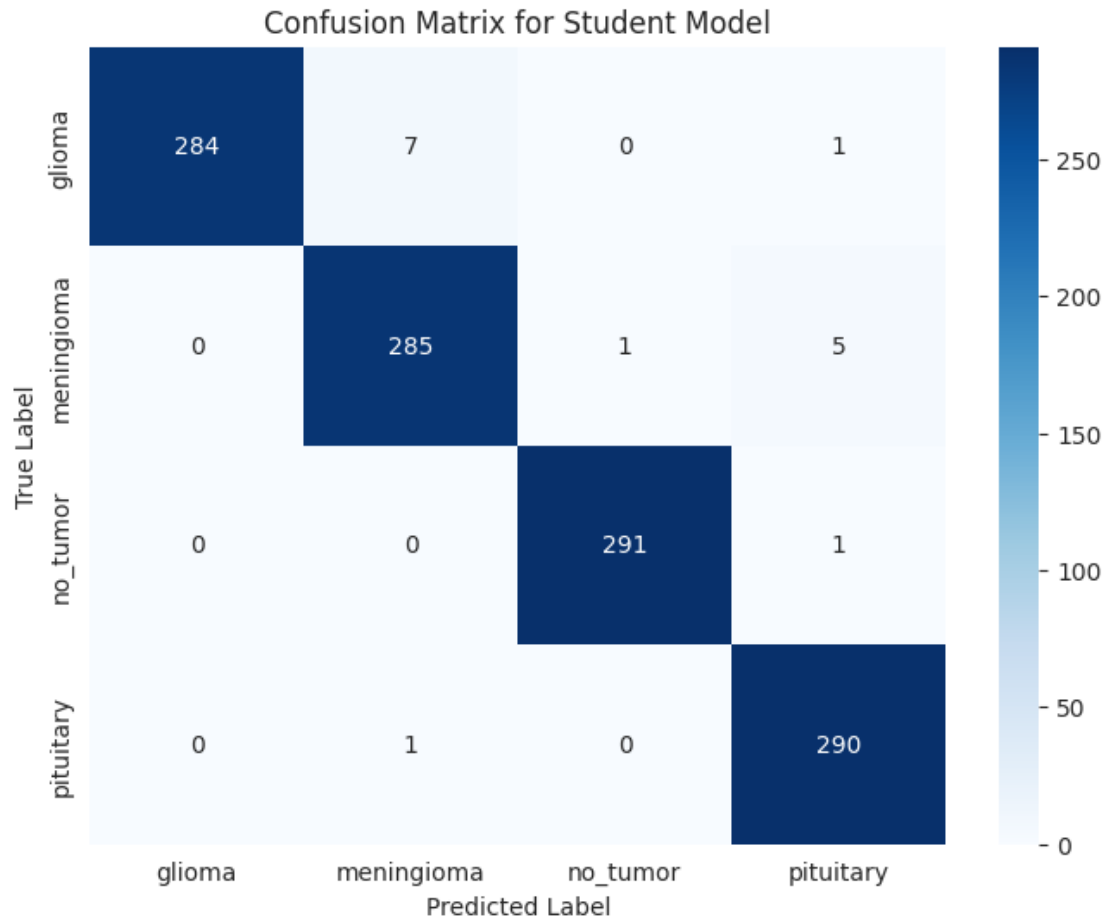
```

Starting JGEKD training for Student model on cuda for 10 epochs.

Epoch 1/10	Total Train Loss: 2.2813	Test Accuracy: 98.03%
Epoch 2/10	Total Train Loss: 2.1748	Test Accuracy: 98.46%
Epoch 3/10	Total Train Loss: 2.1585	Test Accuracy: 98.54%
Epoch 4/10	Total Train Loss: 2.1553	Test Accuracy: 98.63%
Epoch 5/10	Total Train Loss: 2.1503	Test Accuracy: 98.71%
Epoch 6/10	Total Train Loss: 2.1496	Test Accuracy: 98.63%
Epoch 7/10	Total Train Loss: 2.1463	Test Accuracy: 98.71%
Epoch 8/10	Total Train Loss: 2.1457	Test Accuracy: 98.80%
Epoch 9/10	Total Train Loss: 2.1432	Test Accuracy: 98.80%
Epoch 10/10	Total Train Loss: 2.1416	Test Accuracy: 98.63%



=====				
CLASSIFICATION REPORT				
=====				
	precision	recall	f1-score	support
glioma	1.00	0.97	0.99	292
meningioma	0.97	0.98	0.98	291
no_tumor	1.00	1.00	1.00	292
pituitary	0.98	1.00	0.99	291
accuracy			0.99	1166
macro avg	0.99	0.99	0.99	1166
weighted avg	0.99	0.99	0.99	1166



```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns

class JGEKDLoss(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, teacher_logits, student_logits):
        teacher_probs = torch.softmax(teacher_logits, dim=1)
        student_probs = torch.softmax(student_logits, dim=1)
```

```

        teacher_joint_graph = torch.bmm(teacher_probs.unsqueeze(2),
teacher_probs.unsqueeze(1))
        student_joint_graph = torch.bmm(student_probs.unsqueeze(2),
student_probs.unsqueeze(1))

-1)    teacher_flat = teacher_joint_graph.view(teacher_joint_graph.size(0),
-1)    student_flat = student_joint_graph.view(student_joint_graph.size(0),

epsilon = 1e-12
log_student_flat = torch.log(student_flat + epsilon)

kd_loss = -torch.sum(teacher_flat * log_student_flat, dim=1).mean()
return kd_loss

class BrainTumorDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.dataframe = dataframe
        self.transform = transform
        self.class_names = ['glioma', 'meningioma', 'no_tumor', 'pituitary']
        self.label_map = {name: i for i, name in enumerate(self.class_names)}

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, idx):
        img_path = self.dataframe.iloc[idx]['image_path']

        try:
            image = Image.open(img_path).convert('RGB')
        except FileNotFoundError:
            image = Image.fromarray(np.uint8(np.random.rand(224, 224, 3) *
255))

        label_name = self.dataframe.iloc[idx]['label']
        label = self.label_map[label_name]

        if self.transform:
            image = self.transform(image)

        return image, label

def get_data_loaders(df):

    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,

```

```

0.225]])
    ])

    if len(df) < 5:
        train_df = df
        test_df = df
    else:
        train_df, test_df = train_test_split(df, test_size=0.2,
random_state=42, stratify=df['label'])

    train_dataset = BrainTumorDataset(train_df, transform)
    test_dataset = BrainTumorDataset(test_df, transform)

    train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

    return train_loader, test_loader, train_dataset.class_names

def train_and_validate(student_model, teacher_model, train_loader,
test_loader, optimizer, ce_loss_fn, kd_loss_fn, lambda_weight, epochs,
device):

    train_losses = []
    test accuracies = []

    for epoch in range(epochs):

        student_model.train()
        running_loss = 0.0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            with torch.no_grad():
                teacher_logits = teacher_model(images)

            student_logits = student_model(images)

            ce_loss = ce_loss_fn(student_logits, labels)
            kd_loss = kd_loss_fn(teacher_logits, student_logits)

            total_loss = ce_loss + lambda_weight * kd_loss

            optimizer.zero_grad()
            total_loss.backward()
            optimizer.step()

            running_loss += total_loss.item() * images.size(0)

```

```

epoch_train_loss = running_loss / len(train_loader.dataset)
train_losses.append(epoch_train_loss)

student_model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = student_model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

epoch_test_accuracy = 100 * correct / total
test accuracies.append(epoch_test_accuracy)

print(f"Epoch {epoch+1}/{epochs}, Total Train Loss:
{epoch_train_loss:.4f}, Test Accuracy: {epoch_test_accuracy:.2f}%")

return train_losses, test accuracies

def evaluate_metrics(student_model, test_loader, device, class_names):

    student_model.eval()
    all_labels = []
    all_preds = []

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = student_model(images)
            _, predicted = torch.max(outputs.data, 1)

            all_labels.extend(labels.cpu().numpy())
            all_preds.extend(predicted.cpu().numpy())

    print("\n" + "="*50)
    print("CLASSIFICATION REPORT")
    print("="*50)
    print(classification_report(all_labels, all_preds,
target_names=class_names, zero_division=0))

    cm = confusion_matrix(all_labels, all_preds)

    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.title('Confusion Matrix for Student Model (VGG-16 Distilled to

```

```

MobileNetV2)')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.show()

def plot_metrics(train_losses, test_accuracies):

    epochs_range = range(1, len(train_losses) + 1)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, train_losses, label='Total Training Loss')
    plt.title('Loss vs. Epoch (JGEKD)')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(epochs_range, test_accuracies, label='Test Accuracy')
    plt.title('Accuracy vs. Epoch')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy (%)')
    plt.legend()

    plt.tight_layout()
    plt.show()

df['label'] = pd.Categorical(df['label'], categories=['glioma', 'meningioma',
'no_tumor', 'pituitary'])

train_loader, test_loader, CLASS_NAMES = get_data_loaders(df)
NUM_CLASSES = len(CLASS_NAMES)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Defining new models: Teacher (VGG-16) and Student (MobileNetV2)...")

teacher_model = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
teacher_model.classifier[-1] = nn.Linear(teacher_model.classifier[-
1].in_features, NUM_CLASSES)
teacher_model.to(DEVICE)
teacher_model.eval()
for param in teacher_model.parameters():
    param.requires_grad = False

student_model =
models.mobilenet_v2(weights=models.MobileNet_V2_Weights.IMAGENET1K_V1)
student_model.classifier[-1] = nn.Linear(student_model.classifier[-
1].in_features, NUM_CLASSES)

```

```

student_model.to(DEVICE)

ce_loss_fn = nn.CrossEntropyLoss()
kd_loss_fn = JGEKDLoss()
optimizer = optim.Adam(student_model.parameters(), lr=1e-4)
lambda_weight = 0.5
EPOCHS = 10

print(f"Starting JGEKD training for Student (MobileNetV2) on {DEVICE} for
{EPOCHS} epochs.")
train_losses, test accuracies = train_and_validate(
    student_model, teacher_model, train_loader, test_loader,
    optimizer, ce_loss_fn, kd_loss_fn, lambda_weight, EPOCHS, DEVICE
)

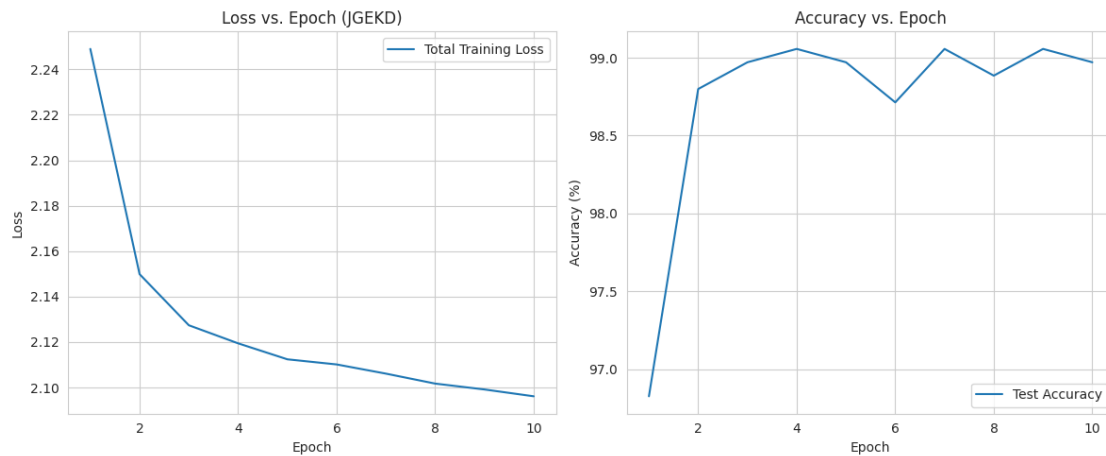
plot_metrics(train_losses, test accuracies)
evaluate_metrics(student_model, test_loader, DEVICE, CLASS_NAMES)

Defining new models: Teacher (VGG-16) and Student (MobileNetV2)...

Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to
/root/.cache/torch/hub/checkpoints/vgg16-397923af.pth
100%|██████████| 528M/528M [00:02<00:00, 224MB/s]
Downloading: "https://download.pytorch.org/models/mobilenet_v2-b0353104.pth"
to /root/.cache/torch/hub/checkpoints/mobilenet_v2-b0353104.pth
100%|██████████| 13.6M/13.6M [00:00<00:00, 119MB/s]

Starting JGEKD training for Student (MobileNetV2) on cuda for 10 epochs.
Epoch 1/10, Total Train Loss: 2.2490, Test Accuracy: 96.83%
Epoch 2/10, Total Train Loss: 2.1499, Test Accuracy: 98.80%
Epoch 3/10, Total Train Loss: 2.1274, Test Accuracy: 98.97%
Epoch 4/10, Total Train Loss: 2.1194, Test Accuracy: 99.06%
Epoch 5/10, Total Train Loss: 2.1124, Test Accuracy: 98.97%
Epoch 6/10, Total Train Loss: 2.1102, Test Accuracy: 98.71%
Epoch 7/10, Total Train Loss: 2.1061, Test Accuracy: 99.06%
Epoch 8/10, Total Train Loss: 2.1017, Test Accuracy: 98.89%
Epoch 9/10, Total Train Loss: 2.0991, Test Accuracy: 99.06%
Epoch 10/10, Total Train Loss: 2.0961, Test Accuracy: 98.97%

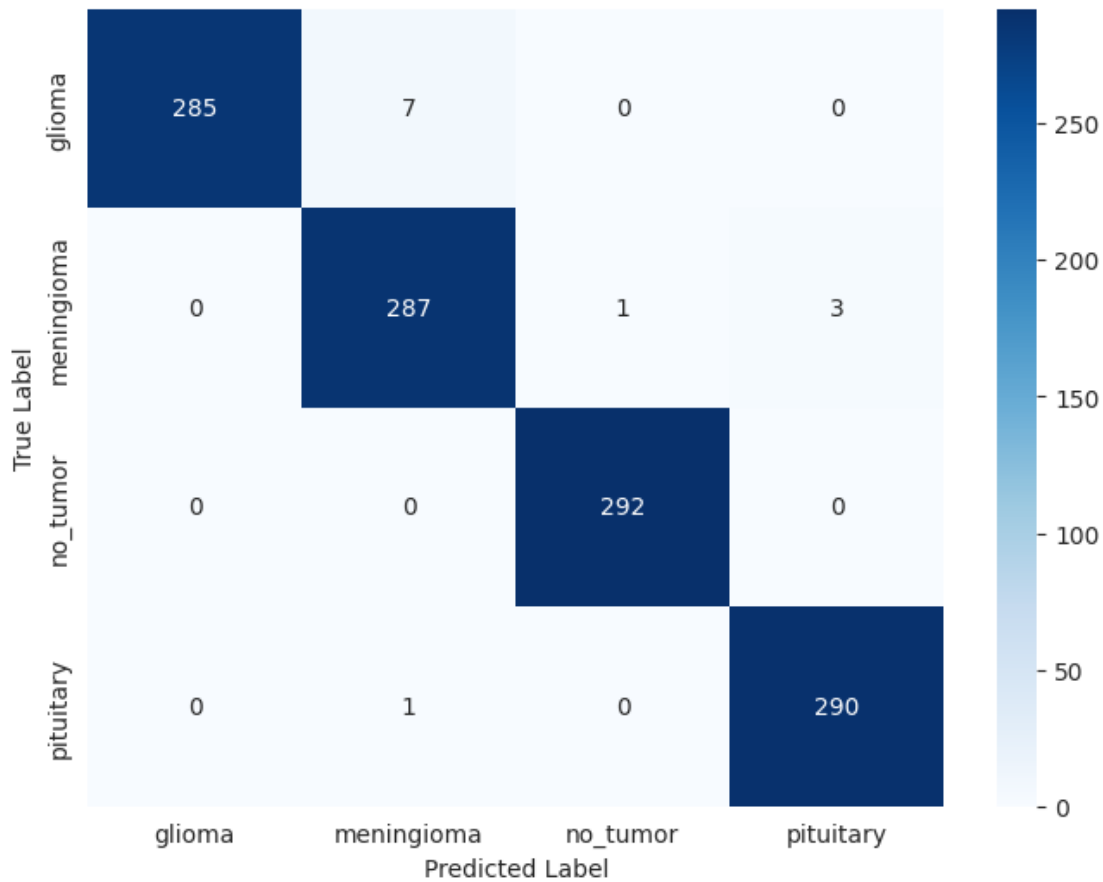
```



CLASSIFICATION REPORT

	precision	recall	f1-score	support
glioma	1.00	0.98	0.99	292
meningioma	0.97	0.99	0.98	291
no_tumor	1.00	1.00	1.00	292
pituitary	0.99	1.00	0.99	291
accuracy			0.99	1166
macro avg	0.99	0.99	0.99	1166
weighted avg	0.99	0.99	0.99	1166

Confusion Matrix for Student Model (VGG-16 Distilled to MobileNetV2)



```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns

class JGEKDLoss(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, teacher_logits, student_logits):
        teacher_probs = torch.softmax(teacher_logits, dim=1)
        student_probs = torch.softmax(student_logits, dim=1)
```

```

        teacher_joint_graph = torch.bmm(teacher_probs.unsqueeze(2),
teacher_probs.unsqueeze(1))
        student_joint_graph = torch.bmm(student_probs.unsqueeze(2),
student_probs.unsqueeze(1))

-1)    teacher_flat = teacher_joint_graph.view(teacher_joint_graph.size(0),
-1)    student_flat = student_joint_graph.view(student_joint_graph.size(0),

        epsilon = 1e-12
        log_student_flat = torch.log(student_flat + epsilon)

        kd_loss = -torch.sum(teacher_flat * log_student_flat, dim=1).mean()
        return kd_loss

class BrainTumorDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.dataframe = dataframe
        self.transform = transform
        self.class_names = ['glioma', 'meningioma', 'no_tumor', 'pituitary']
        self.label_map = {name: i for i, name in enumerate(self.class_names)}

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, idx):
        img_path = self.dataframe.iloc[idx]['image_path']

        try:
            image = Image.open(img_path).convert('RGB')
        except FileNotFoundError:
            image = Image.fromarray(np.uint8(np.random.rand(299, 299, 3) *
255))

        label_name = self.dataframe.iloc[idx]['label']
        label = self.label_map[label_name]

        if self.transform:
            image = self.transform(image)

        return image, label

def get_data_loaders(df):

    transform = transforms.Compose([
        transforms.Resize((299, 299)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,

```

```

0.225]])
    ])

    if len(df) < 5:
        train_df = df
        test_df = df
    else:
        train_df, test_df = train_test_split(df, test_size=0.2,
random_state=42, stratify=df['label'])

    train_dataset = BrainTumorDataset(train_df, transform)
    test_dataset = BrainTumorDataset(test_df, transform)

    train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

    return train_loader, test_loader, train_dataset.class_names

def train_and_validate(student_model, teacher_model, train_loader,
test_loader, optimizer, ce_loss_fn, kd_loss_fn, lambda_weight, epochs,
device):

    train_losses = []
    test accuracies = []

    for epoch in range(epochs):
        student_model.train()
        running_loss = 0.0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            with torch.no_grad():
                teacher_logits = teacher_model(images)

            student_output = student_model(images)

            if isinstance(student_output, tuple):
                student_logits = student_output[0]
                aux_logits = student_output[1]
                ce_loss = ce_loss_fn(student_logits, labels) + 0.4 *
ce_loss_fn(aux_logits, labels) # Standard Inception Loss
            else:
                student_logits = student_output
                ce_loss = ce_loss_fn(student_logits, labels)

            kd_loss = kd_loss_fn(teacher_logits, student_logits)

            total_loss = ce_loss + lambda_weight * kd_loss

```

```

optimizer.zero_grad()
total_loss.backward()
optimizer.step()

running_loss += total_loss.item() * images.size(0)

epoch_train_loss = running_loss / len(train_loader.dataset)
train_losses.append(epoch_train_loss)

student_model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = student_model(images)

        if isinstance(outputs, tuple):
            outputs = outputs[0]

        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

epoch_test_accuracy = 100 * correct / total
test accuracies.append(epoch_test_accuracy)

print(f"Epoch {epoch+1}/{epochs}, Total Train Loss:
{epoch_train_loss:.4f}, Test Accuracy: {epoch_test_accuracy:.2f}%")

return train_losses, test accuracies

def evaluate_metrics(student_model, test_loader, device, class_names):

    student_model.eval()
    all_labels = []
    all_preds = []

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = student_model(images)

            if isinstance(outputs, tuple):
                outputs = outputs[0]

            _, predicted = torch.max(outputs.data, 1)

            all_labels.extend(labels.cpu().numpy())

```

```

        all_preds.extend(predicted.cpu().numpy())

    print("\n" + "="*50)
    print("CLASSIFICATION REPORT")
    print("="*50)
    print(classification_report(all_labels, all_preds,
target_names=class_names, zero_division=0))

    cm = confusion_matrix(all_labels, all_preds)

    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.title('Confusion Matrix for Student Model (DenseNet-161 Distilled to
InceptionV3)')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.show()

def plot_metrics(train_losses, test accuracies):

    epochs_range = range(1, len(train_losses) + 1)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, train_losses, label='Total Training Loss')
    plt.title('Loss vs. Epoch (JGEKD)')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(epochs_range, test accuracies, label='Test Accuracy')
    plt.title('Accuracy vs. Epoch')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy (%)')
    plt.legend()

    plt.tight_layout()
    plt.show()

df['label'] = pd.Categorical(df['label'], categories=['glioma', 'meningioma',
'no_tumor', 'pituitary'])

train_loader, test_loader, CLASS_NAMES = get_data_loaders(df)
NUM_CLASSES = len(CLASS_NAMES)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

print("Defining new models: Teacher (DenseNet-161) and Student
(InceptionV3)...")

teacher_model =
models.densenet161(weights=models.DenseNet161_Weights.IMAGENET1K_V1)
teacher_model.classifier = nn.Linear(teacher_model.classifier.in_features,
NUM_CLASSES)
teacher_model.to(DEVICE)
teacher_model.eval()
for param in teacher_model.parameters():
    param.requires_grad = False

student_model =
models.inception_v3(weights=models.Inception_V3_Weights.IMAGENET1K_V1,
aux_logits=True)
student_model.AuxLogits.fc =
nn.Linear(student_model.AuxLogits.fc.in_features, NUM_CLASSES)
student_model.fc = nn.Linear(student_model.fc.in_features, NUM_CLASSES)
student_model.to(DEVICE)

ce_loss_fn = nn.CrossEntropyLoss()
kd_loss_fn = JGEKDLoss()
optimizer = optim.Adam(student_model.parameters(), lr=1e-4)
lambda_weight = 0.5
EPOCHS = 10

print(f"Starting JGEKD training for Student (InceptionV3) on {DEVICE} for
{EPOCHS} epochs.")
train_losses, test accuracies = train_and_validate(
    student_model, teacher_model, train_loader, test_loader,
    optimizer, ce_loss_fn, kd_loss_fn, lambda_weight, EPOCHS, DEVICE
)

plot_metrics(train_losses, test accuracies)
evaluate_metrics(student_model, test_loader, DEVICE, CLASS_NAMES)

```

Defining new models: Teacher (DenseNet-161) and Student (InceptionV3)...

```

Downloading: "https://download.pytorch.org/models/densenet161-8d451a50.pth"
to /root/.cache/torch/hub/checkpoints/densenet161-8d451a50.pth
100%|██████████| 110M/110M [00:00<00:00, 210MB/s]
Downloading: "https://download.pytorch.org/models/inception_v3_google-
0cc3c7bd.pth" to /root/.cache/torch/hub/checkpoints/inception_v3_google-
0cc3c7bd.pth
100%|██████████| 104M/104M [00:00<00:00, 200MB/s]

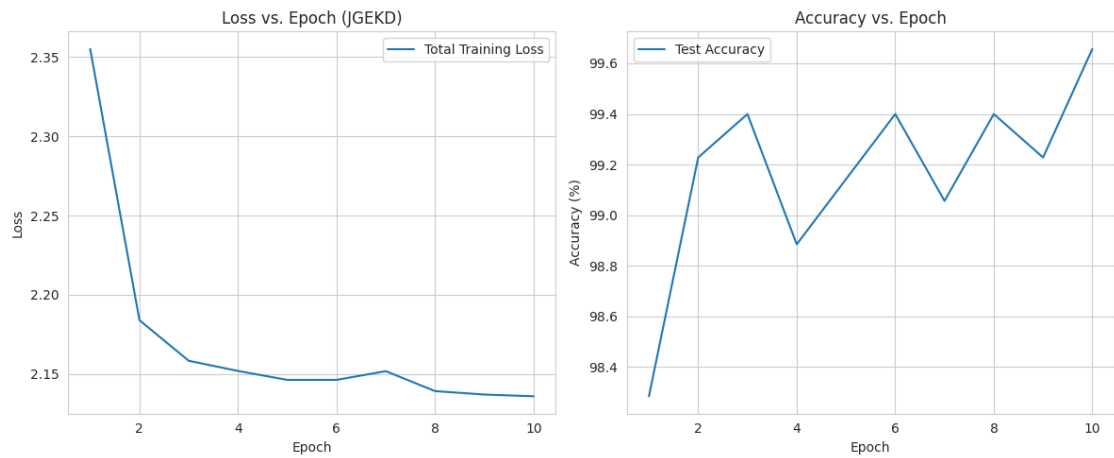
```

```

Starting JGEKD training for Student (InceptionV3) on cuda for 10 epochs.
Epoch 1/10, Total Train Loss: 2.3551, Test Accuracy: 98.28%
Epoch 2/10, Total Train Loss: 2.1840, Test Accuracy: 99.23%
Epoch 3/10, Total Train Loss: 2.1584, Test Accuracy: 99.40%

```

Epoch 4/10, Total Train Loss: 2.1520, Test Accuracy: 98.89%
Epoch 5/10, Total Train Loss: 2.1464, Test Accuracy: 99.14%
Epoch 6/10, Total Train Loss: 2.1464, Test Accuracy: 99.40%
Epoch 7/10, Total Train Loss: 2.1519, Test Accuracy: 99.06%
Epoch 8/10, Total Train Loss: 2.1393, Test Accuracy: 99.40%
Epoch 9/10, Total Train Loss: 2.1372, Test Accuracy: 99.23%
Epoch 10/10, Total Train Loss: 2.1360, Test Accuracy: 99.66%



=====
CLASSIFICATION REPORT
=====

	precision	recall	f1-score	support
glioma	1.00	0.99	1.00	292
meningioma	0.99	1.00	0.99	291
no_tumor	1.00	1.00	1.00	292
pituitary	1.00	1.00	1.00	291
accuracy			1.00	1166
macro avg	1.00	1.00	1.00	1166
weighted avg	1.00	1.00	1.00	1166

Confusion Matrix for Student Model (DenseNet-161 Distilled to InceptionV3)

